

Soluciones de los Ejercicios sobre Tipos Abstractos de Datos (TAD) - Tema 14

Solución al Ejercicio 1: TAD Fecha

```
#include <stdio.h>

typedef struct {
    int dia;
    int mes;
    int anio;
} Fecha;

int es_bisiesto(Fecha f) {
    if ((f.anio % 4 == 0 && f.anio % 100 != 0) || f.anio % 400 == 0) {
        return 1;
    } else {
        return 0;
    }
}

int es_valida(Fecha f) {
    if (f.dia <= 0 || f.mes <= 0 || f.mes > 12 || f.anio < 0) {
        return 0;
    } else {
        int dias_mes[12] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30,
31}; // días totales de cada mes
        if (f.dia <= dias_mes[f.mes]) {
            return 1;
        } else {
            return 0;
        }
    }
}

int main() {
    Fecha f1 = {29, 2, 2024};

    if (es_bisiesto(f1) == 1) {
        printf("Sí es bisiesto\n");
    } else {
        printf("NO es bisiesto\n");
    }

    if (es_valida(f1) == 1) {
        printf("Sí es válida\n");
    } else {
        printf("NO es válida\n");
    }

    return 0;
}
```

Solución al Ejercicio 2: Comparación de Fechas

```
#include <stdio.h>

typedef struct {
    int dia;
    int mes;
    int anio;
} Fecha;

int es_bisiesto(Fecha f) {
    if ((f.anio % 4 == 0 && f.anio % 100 != 0) || f.anio % 400 == 0) {
        return 1;
    } else {
        return 0;
    }
}

int es_valida(Fecha f) {
    if (f.dia <= 0 || f.mes <= 0 || f.mes > 12 || f.anio < 0) {
        return 0;
    } else {
        int dias_mes[12] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31}; // días totales de cada mes
        if (f.dia <= dias_mes[f.mes]) {
            return 1;
        } else {
            return 0;
        }
    }
}

int comparar_fechas(Fecha f1, Fecha f2) {
    if (f1.anio < f2.anio) {
        return 1;
    }
    if (f1.anio > f2.anio) {
        return -1;
    }
    if (f1.mes < f2.mes) {
        return 1;
    }
    if (f1.mes > f2.mes) {
        return -1;
    }
    if (f1.dia < f2.dia) {
        return 1;
    }
    if (f1.dia > f2.dia) {
        return -1;
    }
    return 0;
}

int main() {
    Fecha f1 = {19, 2, 2024};
    Fecha f2 = {29, 2, 2024};
```

```

    if (es_bisiesto(f1) == 1) {
        printf("Sí es bisiesto\n");
    } else {
        printf("NO es bisiesto\n");
    }

    if (es_valida(f1) == 1) {
        printf("Sí es válida\n");
    } else {
        printf("NO es válida\n");
    }

    if (comparar_fechas(f1, f2) == 1) {
        printf("f1 es anterior a f2\n");
    } else if (comparar_fechas(f1, f2) == 0) {
        printf("f1 es igual f2\n");
    } else {
        printf("f1 es posterior a f2\n");
    }

    return 0;
}

```

Solución al Ejercicio 3: Agenda con TAD Cita

```

#include <stdio.h>

#define MAX_CITAS 100

typedef struct {
    int dia;
    int mes;
    int anio;
} Fecha;

typedef struct {
    Fecha fecha;
    int hora;
    char descripcion[100];
} Cita;

int es_bisiesto(Fecha f) {
    if ((f.anio % 4 == 0 && f.anio % 100 != 0) || f.anio % 400 == 0) {
        return 1;
    } else {
        return 0;
    }
}

int es_valida(Fecha f) {
    if (f.dia <= 0 || f.mes <= 0 || f.mes > 12 || f.anio < 0) {
        return 0;
    } else {
        int dias_mes[12] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30,
31}; // días totales de cada mes
    }
}

```

```

        if (f.dia <= dias_mes[f.mes]) {
            return 1;
        } else {
            return 0;
        }
    }
}

int comparar_fechas(Fecha f1, Fecha f2) {
    if (f1.anio < f2.anio) {
        return 1;
    }
    if (f1.anio > f2.anio) {
        return -1;
    }
    if (f1.mes < f2.mes) {
        return 1;
    }
    if (f1.mes > f2.mes) {
        return -1;
    }
    if (f1.dia < f2.dia) {
        return 1;
    }
    if (f1.dia > f2.dia) {
        return -1;
    }
    return 0;
}

bool existe_cita(Cita c, Cita agenda[], int tamano) {
    for (int i = 0; i < tamano; i++) {
        if (comparar_fechas(c.fecha, agenda[i].fecha) == 0 && c.hora ==
agenda[i].hora) {
            return true;
        } else {
            return false;
        }
    }
}

void anadir_cita(Cita c, Cita agenda[], int *tamano) {
    if (*tamano < MAX_CITAS) {
        if (existe_cita(c, agenda, *tamano)) {
            printf("Cita ya existente\n");
        } else {
            int posicion = *tamano;
            agenda[posicion] = c;
            (*tamano)++;
            printf("Se ha introducido la cita\n");
        }
    } else {
        printf("No se ha podido introducir la cita\n");
    }
}

int main() {

```

```

Fecha f1 = {19, 2, 2024};
Fecha f2 = {29, 2, 2024};
Cita agenda[MAX_CITAS];
Cita c = {f1, 22, "Primera cita"};
int tamano = 0;

if (es_bisiesto(f1) == 1) {
    printf("Sí es bisiesto\n");
} else {
    printf("NO es bisiesto\n");
}

if (es_valida(f1) == 1) {
    printf("Sí es válida\n");
} else {
    printf("NO es válida\n");
}

if (comparar_fechas(f1, f2) == 1) {
    printf("f1 es anterior a f2\n");
} else if (comparar_fechas(f1, f2) == 0) {
    printf("f1 es igual f2\n");
} else {
    printf("f1 es posterior a f2\n");
}

anadir_cita(c, agenda, &tamano);

anadir_cita(c, agenda, &tamano);

return 0;
}

```

Solución al Ejercicio 4: Encapsulamiento del TAD Fecha

```

#include <stdio.h>

#define MAX_CITAS 100

typedef struct Fecha {
    void crear_fecha(int d, int m, int a) {
        dia = d;
        mes = m;
        anio = a;
    }

    int obtener_dia() { return dia; }
    int obtener_mes() { return mes; }
    int obtener_anio() { return anio; }

    bool es_bisiesto() {
        return (anio % 4 == 0 and anio % 100 != 0) or (anio % 400 == 0);
    }

    bool es_valida() {
        if (dia <= 0 or mes <= 0 or mes > 12 or anio < 0) {

```

```

        return false;
    }
    int dias_mes[12] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30,
31};
    if (es_bisiesto()) {
        dias_mes[1] = 29;
    }
    return dia <= dias_mes[mes - 1];
}

int comparar(Fecha otra) {
    if (anio < otra.obtener_anio()) return 1;
    if (anio > otra.obtener_anio()) return -1;
    if (mes < otra.obtener_mes()) return 1;
    if (mes > otra.obtener_mes()) return -1;
    if (dia < otra.obtener_dia()) return 1;
    if (dia > otra.obtener_dia()) return -1;
    return 0;
}

private:
    int dia, mes, anio;
};

typedef struct {
    Fecha fecha;
    int hora;
    char descripcion[100];
} Cita;

bool existe_cita(Cita c, Cita agenda[], int tamano) {
    for (int i = 0; i < tamano; i++) {
        if (c.fecha.comparar(agenda[i].fecha) == 0 && c.hora ==
agenda[i].hora) {
            return true;
        }
    }
    return false;
}

void anadir_cita(Cita c, Cita agenda[], int &tamano) {
    if (tamano < MAX_CITAS) {
        if (existe_cita(c, agenda, tamano)) {
            printf("Cita ya existente\n");
        } else {
            agenda[tamano] = c;
            tamano = tamano + 1;
            printf("Se ha introducido la cita\n");
        }
    } else {
        printf("No se ha podido introducir la cita\n");
    }
}

int main() {
    Fecha f1, f2;
    f1.crear_fecha(19, 2, 2024);

```

```

f2.crear_fecha(29, 2, 2024);

Cita agenda[MAX_CITAS];
int tamano = 0;

Cita c = {f1, 22, "Primera cita"};

if (f1.es_bisiesto()) {
    printf("Sí es bisiesto\n");
} else {
    printf("NO es bisiesto\n");
}

if (f1.es_valida()) {
    printf("Sí es válida\n");
} else {
    printf("NO es válida\n");
}

int comp = f1.comparar(f2);
if (comp = 1) {
    printf("f1 es anterior a f2\n");
} else if (comp = 0) {
    printf("f1 es igual a f2\n");
} else {
    printf("f1 es posterior a f2\n");
}

anadir_cita(c, agenda, tamano);
anadir_cita(c, agenda, tamano);

return 0;
}

```